Technická fakulta

Systém pro správu skladu elektronických součástek

Předmět:	Projektování IS
Kód předmětu:	EIT21E
Autoři:	Tomáš Simandl
	Tomáš Szetei
	Lukáš Kvapil
Datum:	27. dubna 2026

Obsah

1 Formulace problému **2**

1.1 Účel 2

1.2 Specifikace 2

2 Datový slovník **3**

2.1 Entity tříd 3

2.2 Atributy 4

3 Objektový model **5**

4 Dynamický model **7**

4.1 Stavový diagram třídy `SkladovyPohyb` 7

4.2 Stavový diagram třídy `Soucastka` 8

5 Model interakcí **9**

5.1 Use Case diagram 9

5.2 Scénáře 10

5.2.1 Scénář: Příjem na sklad (`PrijemPohyb`) 10

5.2.2 Scénář: Výdej ze skladu (`VydejPohyb`) 10

5.2.3 Scénář: Přesun součástky (`PresunutiPohyb`) 10

5.2.4 Scénář: Přiřazení součástky k projektu 11

5.2.5 Scénář: Správa součástek 11

5.3 Sekvenční diagram – Vytvoření pohybu 12

5.4 Aktivity diagram – Vytvoření pohybu 13

6 Závěr **14**

A Zdrojové kódy PlantUML diagramů **15**

A.1 Use Case diagram 16

A.2 Class diagram 18

A.3 Stavový diagram – SkladovyPohyb 21

A.4 Stavový diagram – Soucastka 23

A.5 Sekvenční diagram – Vytvoření pohybu 24

A.6 Diagram aktivit – Vytvoření pohybu 26

1. Formulace problému

1.1 Účel

Cílem tohoto projektu je teoreticky navrhnout informační systém pro správu skladových součástek. Systém má umožňovat evidenci součástek, jejich kategorií, dodavatelů a skladových lokací. Dále má podporovat sledování pohybů součástek ve skladu, jako je naskladnění, vyskladnění nebo přesun mezi jednotlivými lokacemi.

Součástí systému je také možnost přiřazovat součástky k projektům a evidovat jejich množství potřebné pro daný projekt.

1.2 Specifikace

Návrh systému je znázorněn pomocí UML diagramů, které popisují jednotlivé třídy, jejich atributy, vztahy mezi nimi a chování systému při klíčových operacích. Systém zahrnuje objektový model (class diagram), dynamický model (stavové diagramy) a model interakcí (use case diagram, scénáře, sekvenční a aktivity diagram).

2. Datový slovník

2.1 Entity tříd

- **Soucastka**: Skladová položka evidovaná v systému WMS s aktuálním množstvím na skladě. Atribut `rezervovaneMnozstvi` sleduje, kolik kusů je aktuálně blokováno pro projekty – součástka tak může být *Dostupna* i s částečnou rezervací zároveň. Součástka může nabývat stavů: *Dostupna* (k dispozici, případně s rezervovanou částí), *Vydana* (fyzicky vydána ze skladu) nebo *Odepsana* (trvale vyřazena z evidence).
- **Projekt**: Projekt, ke kterému jsou přiřazeny součástky pro sledování spotřeby.
- **SkladovyPohyb**: Abstraktní třída reprezentující pohyb součástek – příjem, výdej nebo přesun.
- **PrijemPohyb**: Konkrétní pohyb typu příjem – naskladnění součástek od dodavatele (dědí ze **SkladovyPohyb**).
- **VydejPohyb**: Konkrétní pohyb typu výdej – vyskladnění součástek ze skladu (dědí ze **SkladovyPohyb**). Atribut `duvod` popisuje účel výdeje, např. *použití v projektu*, *reklamace*, *interní spotřeba* nebo *oprava*.
- **PresunutiPohyb**: Konkrétní pohyb typu přesun – přemístnění součástek mezi lokacemi ve skladu (dědí ze **SkladovyPohyb**).
- **Sklad**: Fyzický sklad rozdělený na lokace, ve kterých jsou uloženy součástky.
- **Lokace**: Konkrétní místo v skladu identifikované označením (např. A-01-03).
- **Uzivatel**: Osoba spravující součástky, pohyby, projekty a sklady v systému.

2.2 Atributy

Tabulka 2.1: Přehled atributů tříd

Entita	**Atributy**
Soucastka	id, nazev, mnozstvi, rezervovaneMnozstvi, stav (Dostupna | Vydana | Odepsana)
Projekt	id, nazev
SkladovyPohyb	id, datum, stav, typ (prijem | vydej | presun), mnozstvi
PrijemPohyb	dodavatel – vlastní; id, datum, stav, typ – zděděno
VydejPohyb	duvod – vlastní; id, datum, stav, typ – zděděno
PresunutiPohyb	zdrojLokaceId: int, cilLokaceId: int – vlastní; id, datum, stav, typ – zděděno
Sklad	id, nazev
Lokace	id, oznaceni
Uzivatel	id, jmeno, email

3. Objektový model

Diagram zachycuje strukturu systému WMS a vztahy mezi jeho hlavními třídami. Součástky jsou přiřazovány k projektům a fyzicky umístěny v lokacích skladu. Třída `Uzivatel` reprezentuje uživatelskou roli v systému. Vytvoření pohybu provádí voláním `SkladovyPohyb.vytvoritPohyb()` a přiřazení součástky k projektu voláním `Projekt.priradSoucastku()`.

Diagram využívá **generalizaci**: abstraktní třída `SkladovyPohyb` je základem pro tři konkrétní podtřídy `PrijemPohyb` (naskladnění), `VydejPohyb` (vyskladnění) a `PresunutiPohyb` (přesun mezi lokacemi), které dědí společné atributy a přidávají vlastní specifické atributy.

Třída `SkladovyPohyb` deklaruje abstraktní metodu `vytvoritPohyb()`, kterou každá podtřída implementuje po svém:

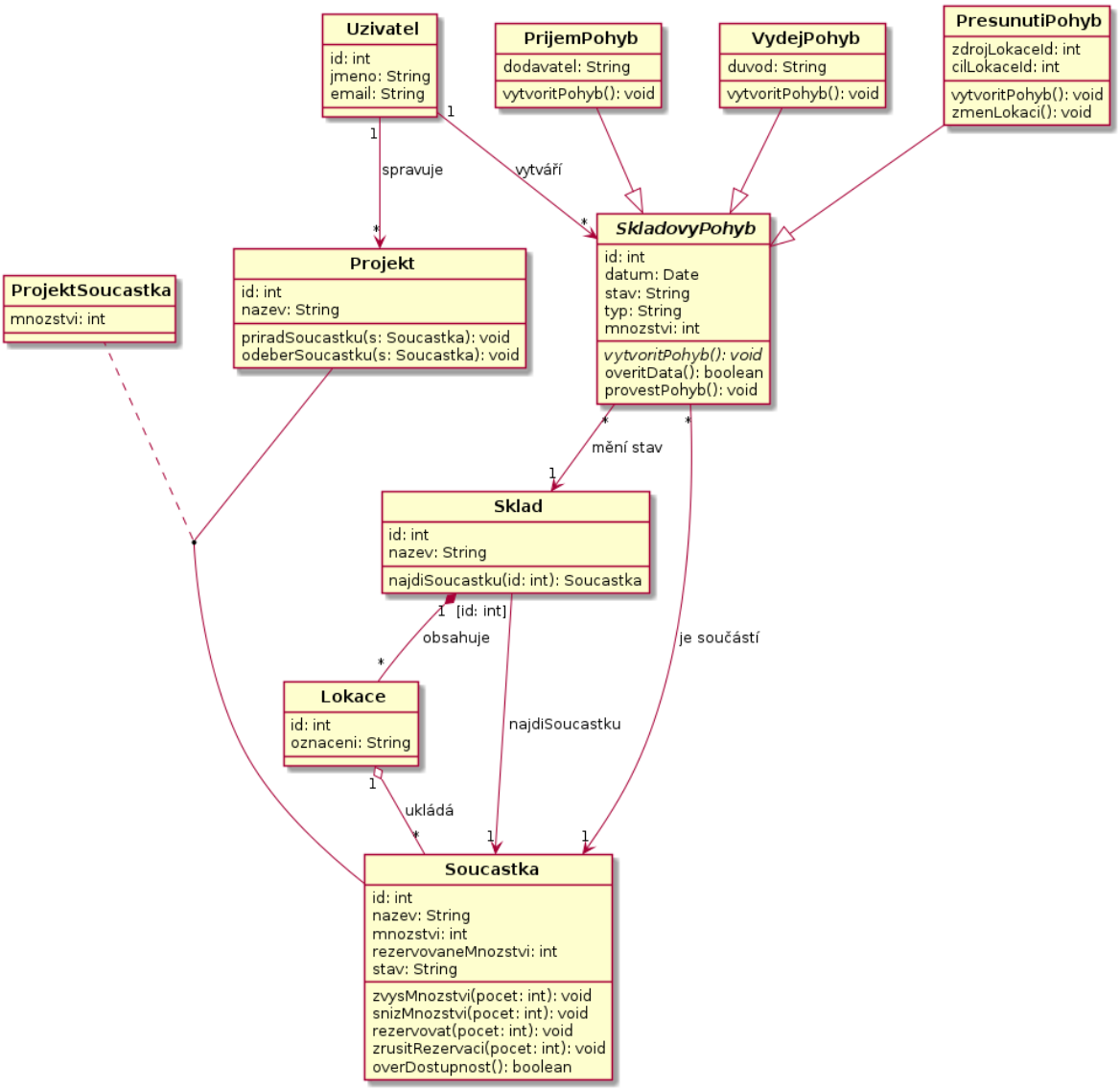
- `PrijemPohyb.vytvoritPohyb()` – vytvoří příjem a nastaví atribut `dodavatel`.
- `VydejPohyb.vytvoritPohyb()` – vytvoří výdej a nastaví atribut `duvod`.
- `PresunutiPohyb.vytvoritPohyb()` – vytvoří přesun a nastaví `zdrojLokaceId` a `cilLokaceId`.

Třída `SkladovyPohyb` zaznamenává každou změnu stavu zásob v systému. Konkrétně:

- `PrijemPohyb` – příjem součástek na sklad od dodavatele; zvyšuje množství dané součástky.
- `VydejPohyb` – výdej součástek ze skladu na projekt; snižuje množství dané součástky.
- `PresunutiPohyb` – přemístnění součástek mezi lokácemi ve skladu; mění přiřazenou lokaci součástky.

Každý pohyb se váže na konkrétní součástku (vazba `je součástí`, multiplicita $* \rightarrow 1$) a mění stav jednoho skladu (vazba `mění stav`, multiplicita $* \rightarrow 1$). Jeden uživatel může vytvořit libovolný počet pohybů (vazba `vytváří`, multiplicita $1 \rightarrow *$). Pohyb nemění fyzickou lokaci součástky – ta je pevně dána vazbou `Lokace` $\rightarrow$ `Soucastka`.

Vazba mezi `Sklad` a `Soucastka` je **kvalifikovaná** – sklad přistupuje ke konkrétní součástce prostřednictvím kvalifikátoru `id: int`.



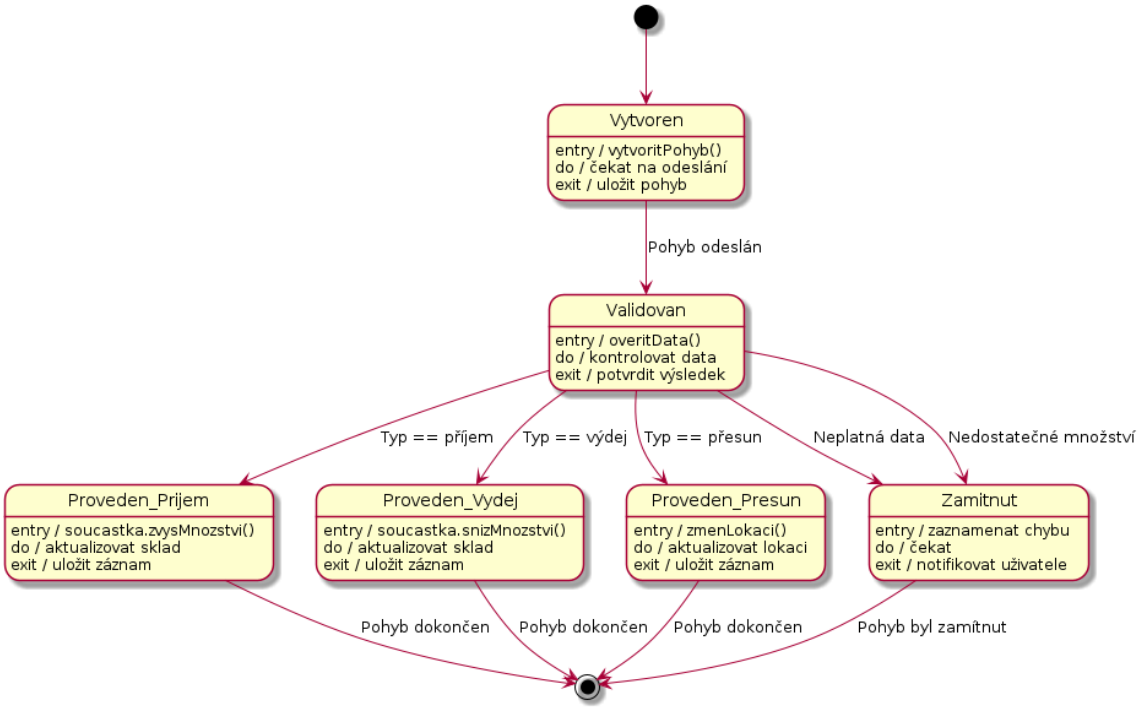
Obrázek 3.1: Class diagram

4. Dynamický model

4.1 Stavový diagram třídy `SkladovyPohyb`

Třída `SkladovyPohyb` reprezentuje obecný skladový pohyb – tedy jakoukoli změnu stavu zásob součástek v systému. Každý pohyb prochází životním cyklem řízeným třemi metodami:

- `vytvoritPohyb()` – inicializuje nový pohyb: přiřadí ID, nastaví datum vzniku, typ pohybu a vazbu na součástku;
- `overitData()` – ověří platnost zadaných dat: existenci součástky, vyplnění povinných polí, dostatečné množství na skladě (příjem/výdej) nebo platnost lokací (přesun);
- `provestPohyb()` – fyzicky provede pohyb podle typu:
  - příjem → `soucastka.zvysMnozstvi(pocet)`
  - výdej → `soucastka.snizMnozstvi(pocet)`
  - přesun → `zmenLokaci()`

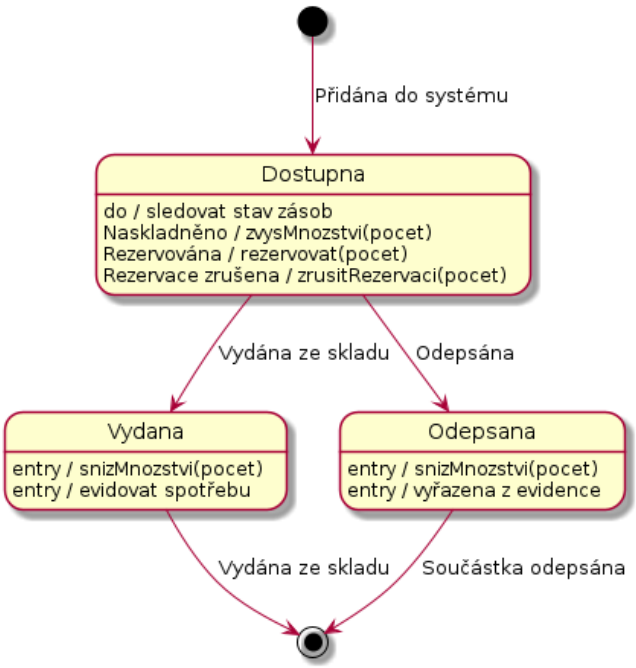


Obrázek 4.1: Stavový diagram třídy SkladovyPohyb

4.2 Stavový diagram třídy `Soucastka`

Diagram zachycuje, jak se mění stav součástky v průběhu skladových operací. Součástka je přidána do systému a okamžitě vstupuje do stavu *Dostupna* s nulovým množstvím. Stav *Dostupna* je hlavní pracovní stav součástky – může mít `mnozstvi = 0` (čeká na naskladnění) i nenulové množství, přičemž část může být blokována atributem `rezervovaneMnozstvi`. Metoda `overDostupnost()` ověří, zda je volné množství (`mnozstvi - rezervovaneMnozstvi > 0`). Stav součástky je řízen těmito metodami:

- `zvysMnozstvi(pocet)` – zavolá se při naskladnění; zvýší atribut `mnozstvi`, součástka zůstává ve stavu *Dostupna*.
- `rezervovat(pocet)` – zablokuje část kusů pro projekt; zvýší `rezervovaneMnozstvi`, stav se nemění.
- `zrusitRezervaci(pocet)` – uvolní rezervaci; sníží `rezervovaneMnozstvi`, stav se nemění.
- `snizMnozstvi(pocet)` – zavolá se při výdeji ze skladu (*Vydana*) nebo odpisu (*Odepsana*); oba jsou terminální stavy.

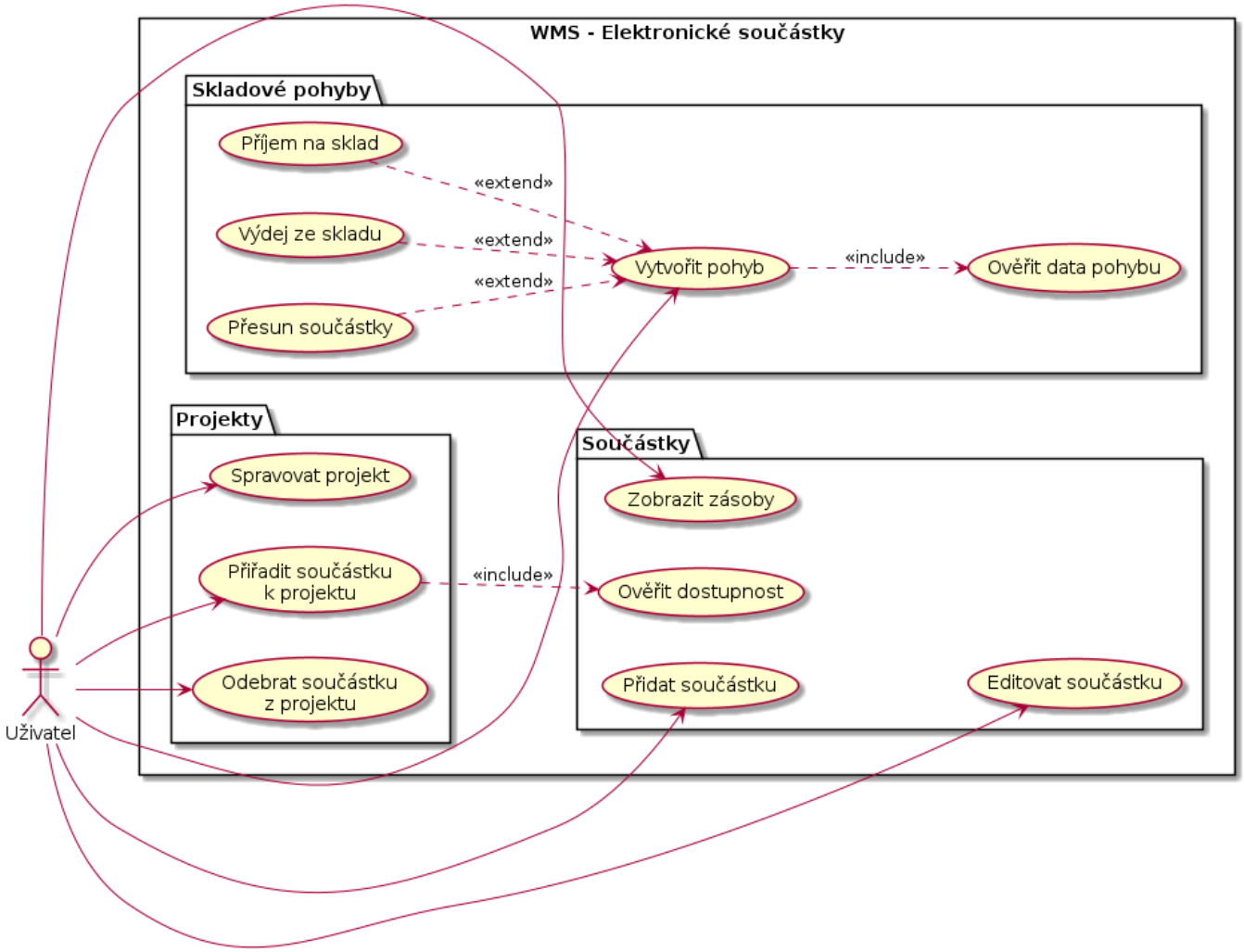


Obrázek 4.2: Stavový diagram třídy Soucastka

5. Model interakcí

5.1 Use Case diagram

Hlavní aktér systému je **Uživatel**, který odpovídá třídě `Uzivatel` v datovém modelu. Uživatel provádí veškeré operace se systémem – vytváří skladové pohyby, spravuje součástky i projekty.



Obrázek 5.1: Use Case diagram

5.2 Scénáře

5.2.1 Scénář: Příjem na sklad (`PrijemPohyb`)

1. Uživatel zvolí typ pohybu *příjem* a zadá součástku, množství a dodavatele.
2. Systém volá `overitData()` – ověří existenci součástky a platnost cílové lokace.
3. Pokud data OK → systém volá `provestPohyb()`, který interně volá `Soucastka.zvysMnozstvi`
4. Pohyb je potvrzen a zásoby jsou aktualizovány.
5. **Alternativní tok:** součástka neexistuje nebo lokace je neplatná → systém vrátí chybovou hlášku a pohyb je zamítnut.

5.2.2 Scénář: Výdej ze skladu (`VydejPohyb`)

1. Uživatel zvolí typ pohybu *výdej* a zadá součástku, množství a důvod výdeje.
2. Systém volá `overitData()` – ověří existenci součástky a zda `mnozstvi` $\geq$ požadovanému množství.
3. Pokud data OK → systém volá `provestPohyb()`, který interně volá `Soucastka.snizMnozstvi`
4. Pohyb je potvrzen a zásoby jsou sníženy.
5. **Alternativní tok:** nedostatečné množství na skladě → systém vrátí chybovou hlášku a pohyb je zamítnut.

5.2.3 Scénář: Přesun součástky (`PresunutiPohyb`)

1. Uživatel zvolí typ pohybu *přesun* a zadá součástku, množství, zdrojovou a cílovou lokaci.
2. Systém volá `overitData()` – ověří existenci součástky, dostatečné množství ve zdrojové lokaci a platnost cílové lokace.
3. Pokud data OK → systém volá `provestPohyb()` a poté `PresunutiPohyb.zmenLokaci()`.
4. Pohyb je potvrzen; součástka je evidována v cílové lokaci.
5. **Alternativní tok:** zdrojová nebo cílová lokace je neplatná → systém vrátí chybovou hlášku a pohyb je zamítnut.

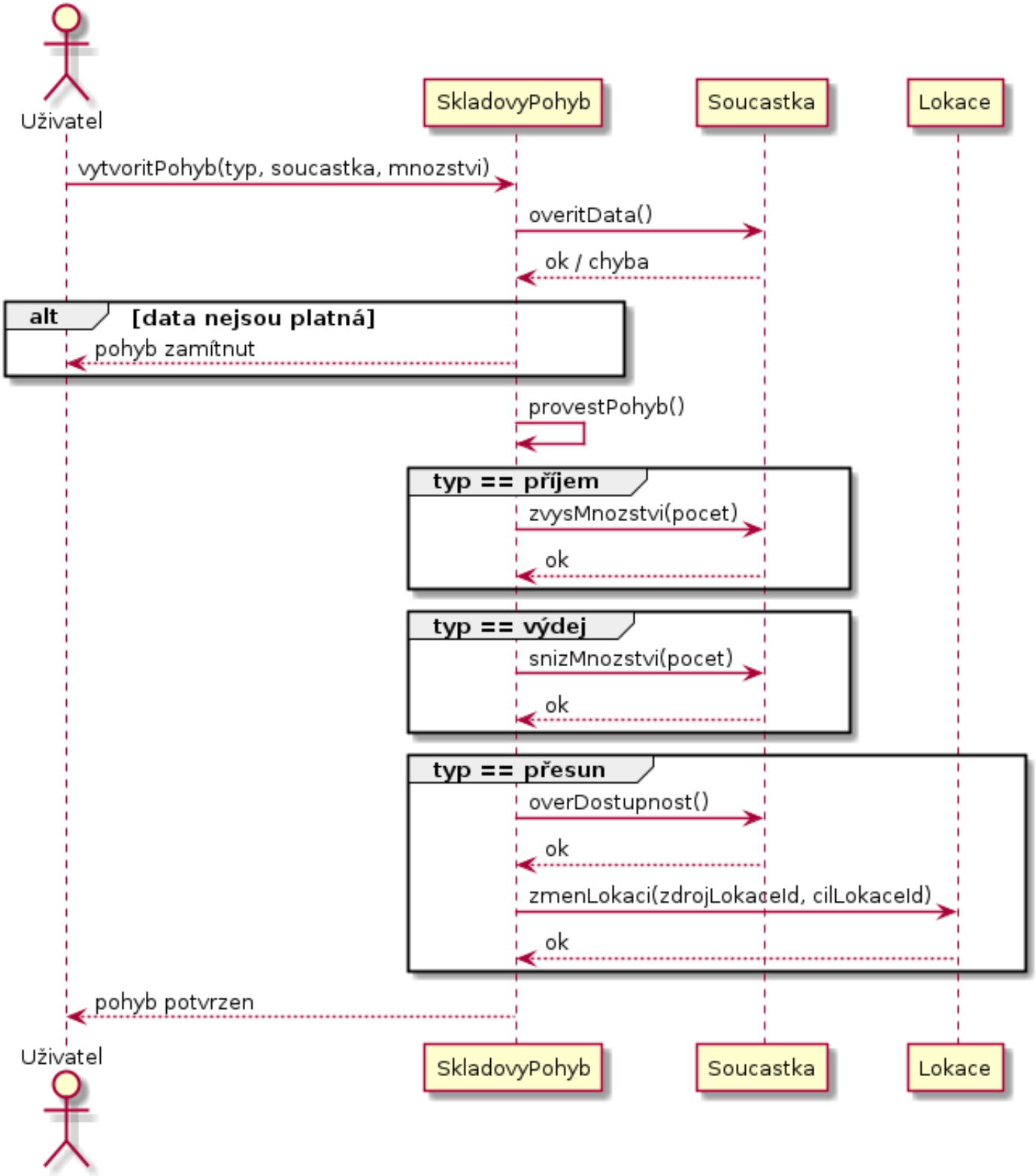
5.2.4 Scénář: Přiřazení součástky k projektu

1. Uživatel vybere projekt a zadá součástku s množstvím.
2. Systém volá `Soucastka.overDostupnost()` – ověří, zda `mnozstvi` − `rezervovaneMnozstvi` > 0.
3. Pokud je dostatek volných kusů → systém volá `Projekt.priradSoucastku()`, které interně zavolá `Soucastka.rezervovat()`.
4. Součástka je přiřazena k projektu a rezervované množství je aktualizováno.
5. **Alternativní tok:** nedostatečný počet volných kusů → systém vrátí chybovou hlášku a přiřazení je zamítnuto.

5.2.5 Scénář: Správa součástek

1. Uživatel přidá novou součástku nebo upraví existující.
2. Systém ověří validitu zadaných dat (název, počáteční množství, lokace).
3. Pokud data OK → součástka je uložena; zásoby jsou aktualizovány.
4. **Alternativní tok:** neplatná data → systém vrátí chybovou hlášku, součástka není uložena.

5.3 Sekvenční diagram – Vytvoření pohybu

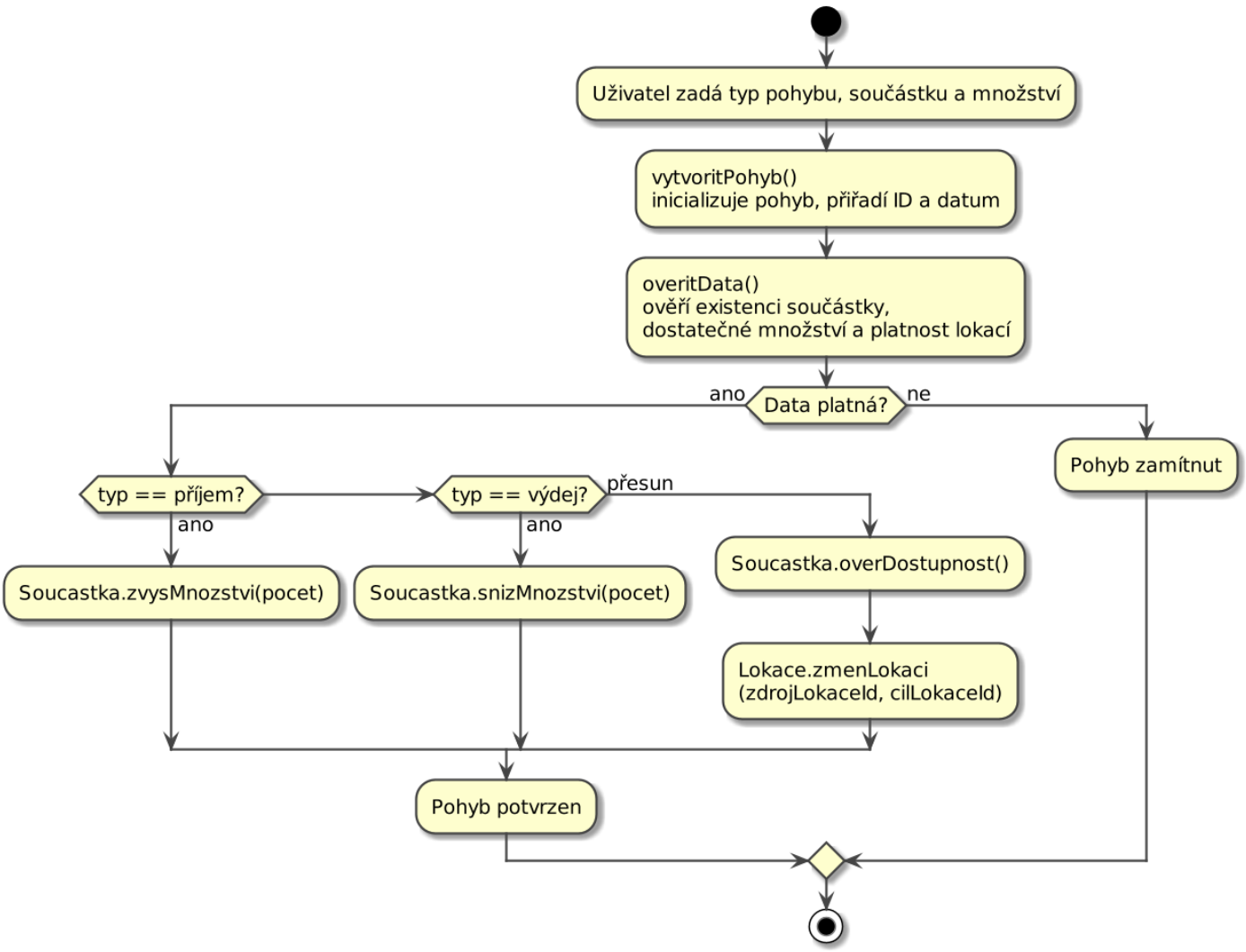


Obrázek 5.2: Sekvenční diagram – Vytvoření pohybu

5.4 Aktivity diagram – Vytvoření pohybu

Diagram zachycuje průběh vytvoření skladového pohybu v systému. Pohyb prochází třemi fázemi řízenými metodami třídy `SkladovyPohyb`:

- `vytvoritPohyb()` – uživatel zadá typ pohybu, součástku a množství; metoda inicializuje pohyb, přiřadí ID a datum vzniku.
- `overitData()` – ověří platnost zadaných dat: existenci součástky, dostatečné množství na skladě a platnost lokací u přesunu.
- `provestPohyb()` – fyzicky provede pohyb podle typu: příjem zavolá `zvysMnozstvi(pocet)`, výdej `snizMnozstvi(pocet)`, přesun `zmenLokaci()`. Pokud data nejsou platná, pohyb je zamítnut a uložena chyba.



Obrázek 5.3: Aktivity diagram – Vytvoření pohybu

6. Závěr

Navržený datový model systému zachycuje strukturu, chování i interakce hlavních entit systému. Objektový model (class diagram) definuje třídy a jejich vztahy. Dynamický model (stavové diagramy) popisuje životní cyklus tříd `SkladovyPohyb` a `Soucastka`. Model interakcí (Use Case diagram, scénáře, sekvenční a aktivity diagram) zachycuje komunikaci mezi aktérem a systémem při klíčových operacích.

A. Zdrojové kódy PlantUML diagramů

Tato příloha obsahuje zdrojové kódy všech UML diagramů vygenerovaných nástrojem PlantUML.

A.1 Use Case diagram

```
@startuml 01_use_case
left to right direction

actor "Uživatel" as Uzivatel

rectangle "WMS     Elektronické součástky" {

  package "Skladové pohyby" {
    usecase "Vytvořit pohyb" as UC_POHYB
    usecase "Ověřit data pohybu" as UC_OVERIT
    usecase "Příjem na sklad" as UC_PRIJEM
    usecase "V dej ze skladu" as UC_VYDEJ
    usecase "Přesun součástky" as UC_PRESUN
  }

  package "Součástky" {
    usecase "Přidat součástku" as UC_ADD
    usecase "Editovat součástku" as UC_EDIT
    usecase "Zobrazit zásoby" as UC_ZASOBY
    usecase "Ověřit dostupnost" as UC_DOSTUPNOST
    UC_ADD -[hidden]down- UC_EDIT
  }

  package "Projekty" {
    usecase "Spravovat projekt" as UC_PROJEKT
    usecase "Přiřadit součástku\nk projektu" as UC_PRIRADIT
    usecase "Odebrat součástku\nz projektu" as UC_ODEBRAT
  }

}

Uzivatel --> UC_POHYB
Uzivatel --> UC_ADD
Uzivatel --> UC_EDIT
Uzivatel --> UC_ZASOBY
Uzivatel --> UC_PROJEKT
Uzivatel --> UC_PRIRADIT
Uzivatel --> UC_ODEBRAT

```

```
40 ' include: od zahrnujiciho k zahrnutemu (povinne)
41 UC_POHYB    ..> UC_OVERIT     : <<include>>
42 UC_PRIRADIT ..> UC_DOSTUPNOST : <<include>>
43
44 ' extend: od pridaneho k puvodnimu (specializace)
45 UC_PRIJEM ..> UC_POHYB : <<extend>>
46 UC_VYDEJ  ..> UC_POHYB : <<extend>>
47 UC_PRESUN ..> UC_POHYB : <<extend>>
48
49 @enduml
```

A.2 Class diagram

```
@startuml
top to bottom direction
skinparam CircledCharacterRadius 0
skinparam CircledCharacterFontSize 0
skinparam defaultFontSize 13
skinparam classFontSize 14
skinparam classFontStyle bold
skinparam ArrowFontSize 12
skinparam nodesep 50
skinparam ranksep 60

class Uzivatel {
  id: int
  jmeno: String
  email: String
}

class PrijemPohyb {
  dodavatel: String
  vytvoritPohyb(): void
}

class VydejPohyb {
  duvod: String
  vytvoritPohyb(): void
}

class PresunutiPohyb {
  zdrojLokaceId: int
  cilLokaceId: int
  vytvoritPohyb(): void
  zmenLokaci(): void
}

abstract class SkladovyPohyb {
  id: int
  datum: Date
  stav: String
  typ: String
```

```
  mnozstvi: int
  {abstract} vytvoritPohyb(): void
  overitData(): boolean
  provestPohyb(): void
}

class Projekt {
  id: int
  nazev: String
  priradSoucastku(s: Soucastka): void
  odeberSoucastku(s: Soucastka): void
}

class ProjektSoucastka {
  mnozstvi: int
}

class Sklad {
  id: int
  nazev: String
  najdiSoucastku(id: int): Soucastka
}

class Lokace {
  id: int
  oznaceni: String
}

class Soucastka {
  id: int
  nazev: String
  mnozstvi: int
  rezervovaneMnozstvi: int
  stav: String
  zvysMnozstvi(pocet: int): void
  snizMnozstvi(pocet: int): void
  rezervovat(pocet: int): void
  zrusitRezervaci(pocet: int): void
  overDostupnost(): boolean
}

```

```
'        Generalizace


PrijemPohyb    --|> SkladovyPohyb
VydejPohyb     --|> SkladovyPohyb
PresunutiPohyb --|> SkladovyPohyb

'        Atribut spojení


(Projekt, Soucastka) .. ProjektSoucastka
Projekt -[hidden]left- ProjektSoucastka

'        Kvalifikovaná vazba


Sklad "[id: int]" --> "1" Soucastka : najdiSoucastku

'        Asociace


SkladovyPohyb "*" --> "1" Soucastka     : je součástí
Sklad         "1" *-- "*" Lokace        : obsahuje
Lokace        "1" o-- "*" Soucastka     : ukládá
SkladovyPohyb "*" --> "1" Sklad         : mění stav
Uzivatel      "1" --> "*" SkladovyPohyb : vytváří
Uzivatel      "1" --> "*" Projekt       : spravuje

'        Layout


Uzivatel    -[hidden]right- PrijemPohyb
PrijemPohyb -[hidden]right- VydejPohyb
VydejPohyb  -[hidden]right- PresunutiPohyb

@enduml
```

A.3 Stavový diagram – SkladovyPohyb

```
@startuml
top to bottom direction

[*] --> Vytvoren

Vytvoren : entry / vytvoritPohyb()
Vytvoren : do / čekat na odeslání
Vytvoren : exit / uložit pohyb
Vytvoren --> Validovan : Pohyb odeslán

Validovan : entry / overitData()
Validovan : do / kontrolovat data
Validovan : exit / potvrdit v sledek
Validovan --> Proveden_Prijem : Typ == příjem
Validovan --> Proveden_Vydej : Typ == v dej
Validovan --> Proveden_Presun : Typ == přesun
Validovan --> Zamitnut : Neplatná data
Validovan --> Zamitnut : Nedostatečné množství

Zamitnut : entry / zaznamenat chybu
Zamitnut : do / čekat
Zamitnut : exit / notifikovat uživatele

Proveden_Prijem : entry / soucastka.zvysMnozstvi()
Proveden_Prijem : do / aktualizovat sklad
Proveden_Prijem : exit / uložit záznam

Proveden_Vydej : entry / soucastka.snizMnozstvi()
Proveden_Vydej : do / aktualizovat sklad
Proveden_Vydej : exit / uložit záznam

Proveden_Presun : entry / zmenLokaci()
Proveden_Presun : do / aktualizovat lokaci
Proveden_Presun : exit / uložit záznam

Proveden_Prijem --> [*] : Pohyb dokončen
Proveden_Vydej --> [*] : Pohyb dokončen
Proveden_Presun --> [*] : Pohyb dokončen
Zamitnut --> [*] : Pohyb byl zamítnut
```

```
40
41 @enduml
```

A.4 Stavový diagram – Soucastka

```
@startuml
top to bottom direction


[*] --> Dostupna : Přidána do systému

Dostupna : do / sledovat stav zásob
Dostupna : Naskladněno / zvysMnozstvi(pocet)
Dostupna : Rezervována / rezervovat(pocet)
Dostupna : Rezervace zrušena / zrusitRezervaci(pocet)
Dostupna --> Vydana : Vydána ze skladu
Dostupna --> Odepsana : Odepsána

Vydana : entry / snizMnozstvi(pocet)
Vydana : entry / evidovat spotřebu
Vydana --> [*] : Vydána ze skladu

Odepsana : entry / snizMnozstvi(pocet)
Odepsana : entry / vyřazena z evidence
Odepsana --> [*] : Součástka odepsána

@enduml
```

A.5 Sekvenční diagram – Vytvoření pohybu

```
@startuml
skinparam defaultFontSize 13
skinparam sequenceArrowThickness 1.5
skinparam sequenceParticipantBorderThickness 1.5
skinparam sequenceGroupBorderThickness 1.5
skinparam sequenceBoxBorderColor #888888

actor "Uživatel" as Uzivatel
participant "SkladovyPohyb" as Pohyb
participant "Soucastka" as Soucastka
participant "Lokace" as Lokace

Uzivatel -> Pohyb : vytvoritPohyb(typ, soucastka, mnozstvi)
Pohyb -> Soucastka : overitData()
Soucastka --> Pohyb : ok / chyba

alt data nejsou platná
    Pohyb --> Uzivatel : pohyb zamítnut
end

Pohyb -> Pohyb : provestPohyb()

group typ == příjem
    Pohyb -> Soucastka : zvysMnozstvi(pocet)
    Soucastka --> Pohyb : ok
end

group typ == v dej
    Pohyb -> Soucastka : snizMnozstvi(pocet)
    Soucastka --> Pohyb : ok
end

group typ == přesun
    Pohyb -> Soucastka : overDostupnost()
    Soucastka --> Pohyb : ok
    Pohyb -> Lokace : zmenLokaci(zdrojLokaceId, cilLokaceId)
    Lokace --> Pohyb : ok
end

```

```
Pohyb --> Uzivatel : pohyb potvrzen
@enduml
```

A.6 Diagram aktivit – Vytvoření pohybu

```
@startuml
scale 1.5
skinparam defaultFontSize 13
skinparam ArrowThickness 1.5
skinparam ActivityBorderThickness 1.5
skinparam ActivityDiamondBorderThickness 1.5
skinparam ActivityBorderColor #444444
skinparam ActivityDiamondBorderColor #444444
skinparam ArrowColor #444444

start

:Uživatel zadá typ pohybu, součástku a množství;

:vytvoritPohyb()\ninicializuje pohyb, přiřadí ID a datum;

:overitData()\nověří existenci součástky,\ndostatečné množství
   a platnost lokací;

if (Data platná?) then (ano)

  if (typ == příjem?) then (ano)
    :Soucastka.zvysMnozstvi(pocet);
  elseif (typ == v dej?) then (ano)
    :Soucastka.snizMnozstvi(pocet);
  else (přesun)
    :Soucastka.overDostupnost();
    :Lokace.zmenLokaci\n(zdrojLokaceId, cilLokaceId);
  endif

  :Pohyb potvrzen;

else (ne)
  :Pohyb zamítnut;
endif

stop
@enduml
```